

Quand l'Informatique
s'associe à l'Electronique

Projet C - Electronique

Encadreurs : M. François Pecheux
M. Arouna Darga
M. Rémi Griot

Seynabou NDOUR / Chrisnie MBON

Projet Informatique Electronique

Présentation du projet

Le projet qui nous a été soumis consiste à faire tourner un grafcet, codé en langage C sur une carte électronique en mettant en place un mécanisme de communication et de traduction.

Il s'agit de deux projets combinés en un seul, notamment, un projet informatique dont le but est de parvenir à interpréter un grafcet et un projet électronique qui consiste à faire tourner le grafcet dans un microcontrôleur, dans notre cas, il s'agit de la carte Nucleo STM-32, afin de commander un moteur.

A terme notre projet pourra contribuer à automatiser les tapis roulants pour les chaînes de production des étudiants de la filière AGRAL.

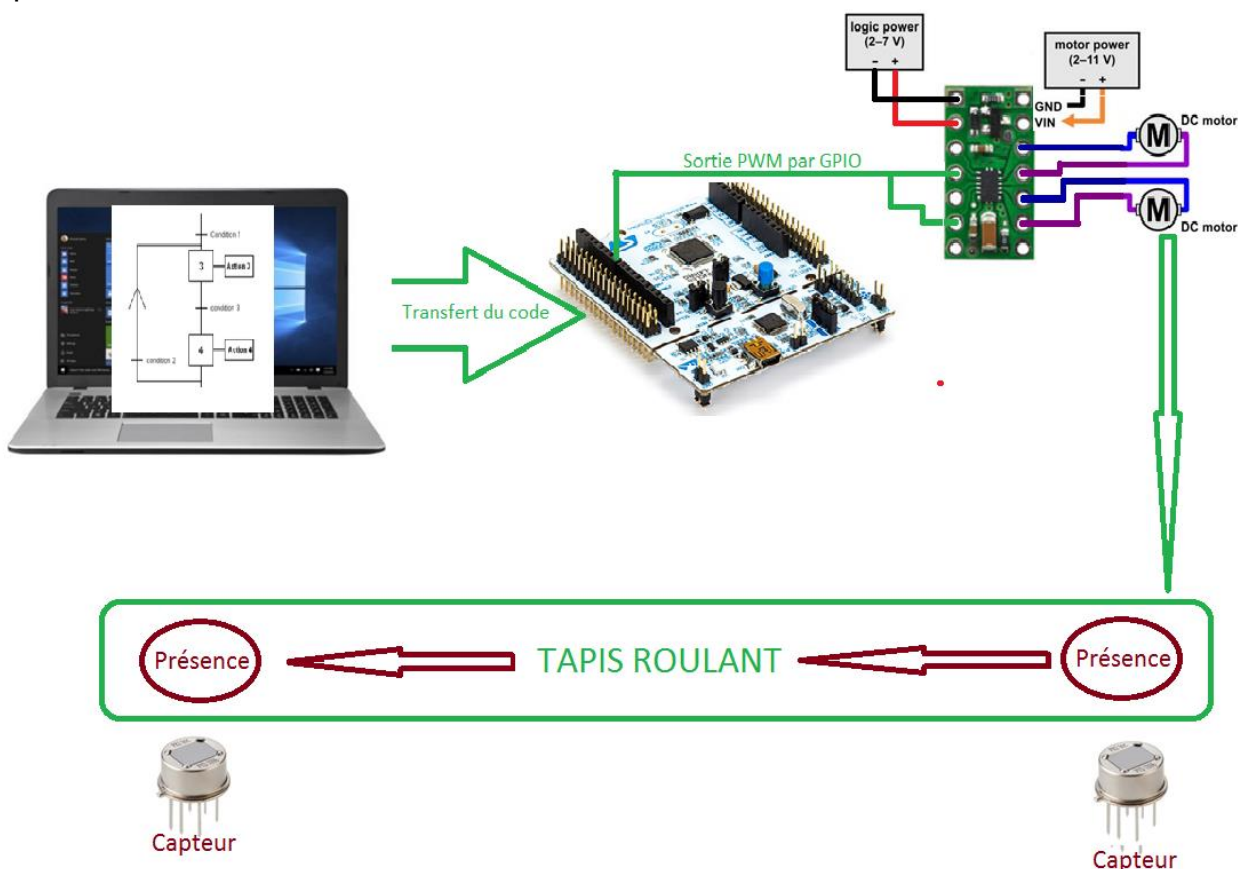


Figure 1 Ensemble du Système

Objectifs

Le choix de ce projet vise un certain nombre d'objectifs, notamment :

- Consolider la maîtrise du langage C ;
- Apprendre à faire du C sur un ordinateur de bureau et aussi sur un microcontrôleur ;
- Fabriquer un « outil logiciel » : le compilateur grafcet ;
- Apprendre à utiliser un protocole de communication : UART.

Projet Informatique Electronique

I- Projet C : Interprétation d'un grafcet

Comme évoqué ci-dessus, le projet C est de réaliser un interpréteur de grafcet. Un grafcet est un graphe fonctionnel de commande des étapes de transitions. Comment cela devrait-il se faire ?

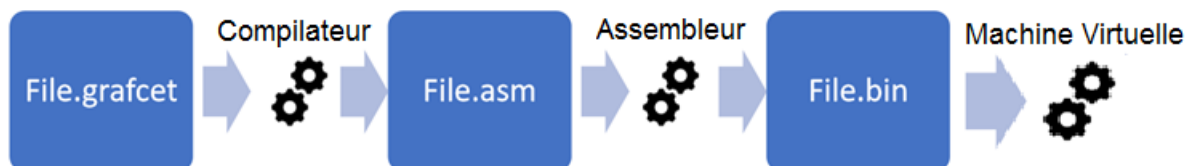


Figure 2 Etapes pour fabriquer un interpréteur de grafcet

Ci-dessus, l'ensemble des étapes nécessaires pour créer un interpréteur de grafcet. Nous avons commencé par la fin de la chaîne, c'est-à-dire la machine virtuelle.

1- La machine virtuelle

La machine virtuelle devra exécuter le bycode (fichier.bin), elle est programmée de sorte à comprendre le langage machine (push, pop, ...), c'est une machine à états qui fait essentiellement des opérations logiques et arithmétiques et stocke les résultats dans une pile ou un tableau.

2- L'assembleur

L'assembleur permet de générer un tableau OP_Code et de le compléter à partir du fichier généré par le compilateur.

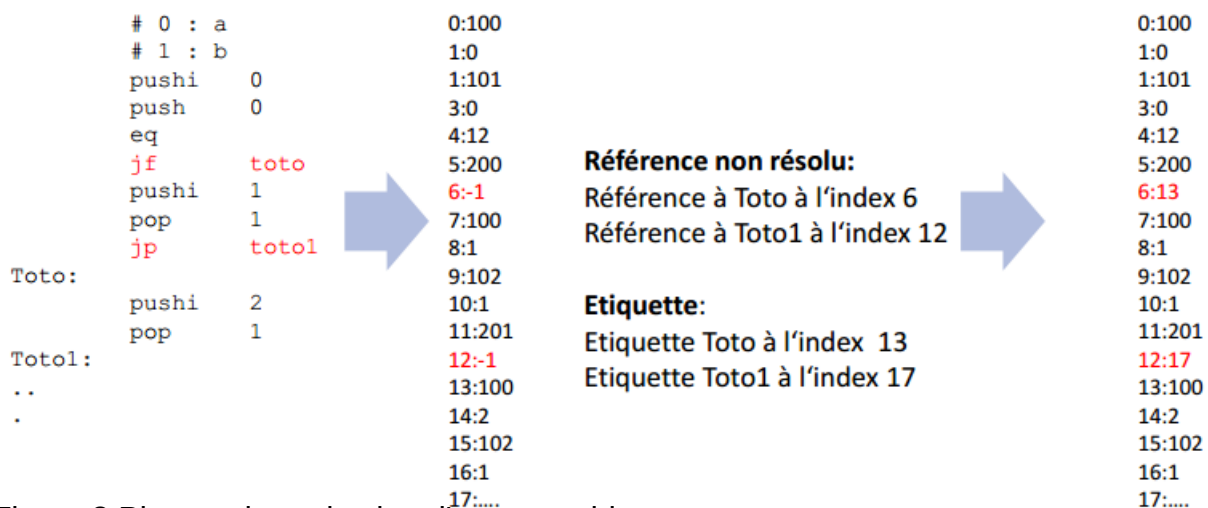


Figure 3 Phases de traduction d'un assembleur

Projet Informatique Electronique

Nous devons compléter des fonctions et les appeler, comme l'indique la figure ci-dessous, dans le fonction main.

```
int main(int argc, char **argv){
    // Ouverture du fichier
    ...
    // Construction du dictionnaire
d'instruction
    addInstructionName(...)
    // Parseur phase 1
    parseAsm(fin);
    // Parseur phase 2
    resolveReferences();
    // Génération du code
    generateBinary(fout);
    return 0;
}
```

3- Le compilateur

Le compilateur devra traduire en langage assembleur, le grafcet que nous lui fournirons. Il effectue une analyse lexicale (prétraitement), et une analyse syntaxique (*parsing*), et construit un arbre syntaxique.

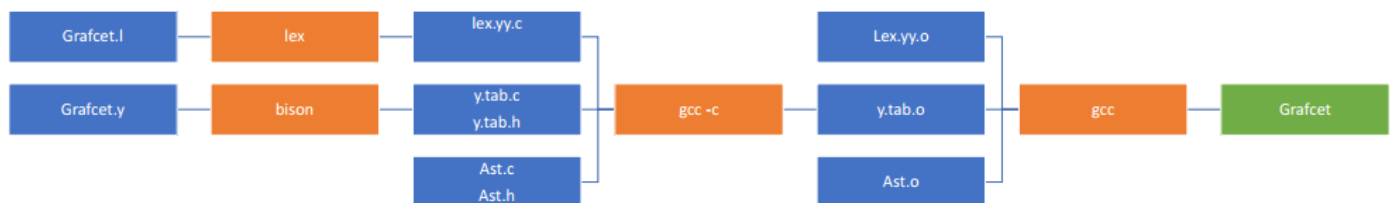


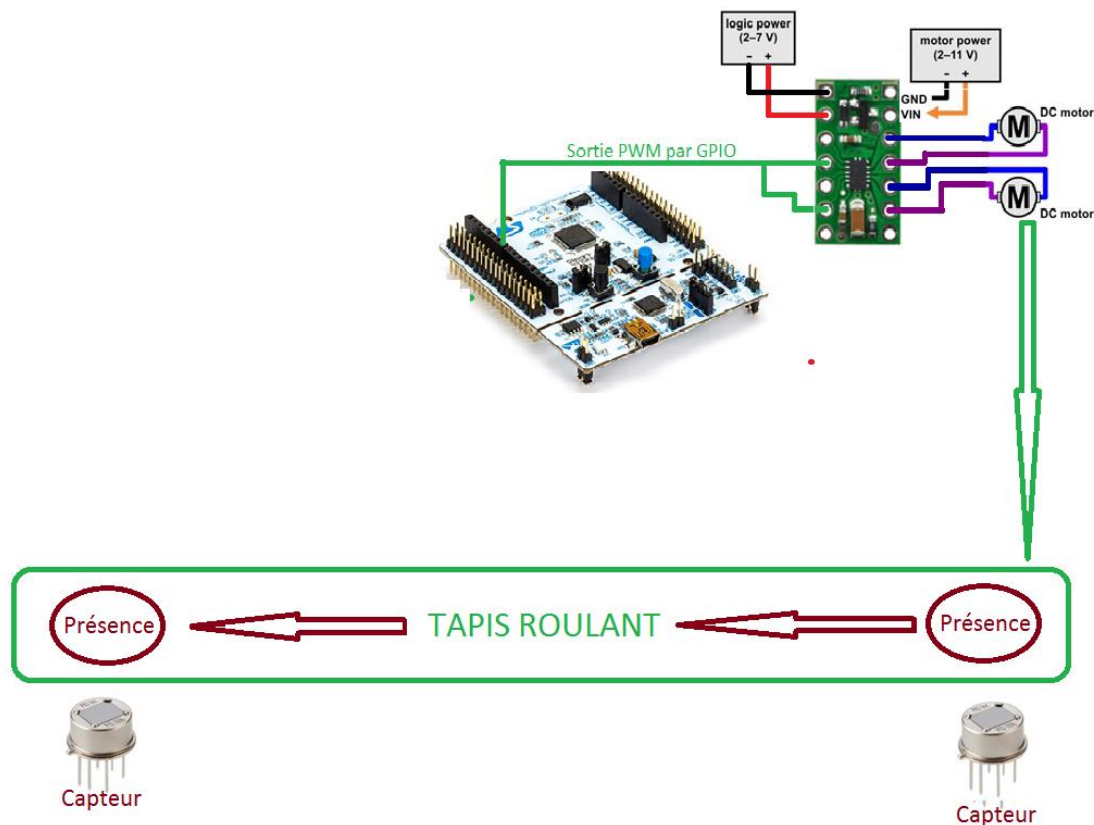
Figure 4 Compilateur

II- Projet Electronique

Dans cette partie nous devons implémenter notre machine dans le microcontrôleur, d'une part. D'autre part, commander le démarrage, la direction et l'arrêt du moteur en fonction des informations envoyées par les capteurs du tapis roulant.

Nous utilisons la carte Nucleo STM32 et le driver DRV-8835 pour piloter le moteur.

Projet Informatique Electronique



4- Bilan du travail réalisé

La machine virtuelle que nous avons créée fonctionne bien, nous pouvons effectuer des calculs, faire des comparaisons, des opérations logiques de type AND et OR.

Nous avons également fait les fichiers `grafcet.asm`, `grafcetInterpreter.c`. Quant à l'assembleur (`asm.c`), nous n'avons malheureusement pas réussi à en créer, nous avons mal géré notre temps pour cette partie, nous étions obligées de passer à la partie électronique et au compilateur en parallèle.

Le compilateur (`ast.c`) avait deux fonctions à compléter la fonction `makedual` et la fonction `generatecode`.

La fonction `makedual`, qui permet de construire l'arbre syntaxique et pour exploiter ce dernier, la fonction `generatecode`, afin de générer le code.

Nous avons rencontré quelques difficultés lors de la programmation du compilateur au niveau des tests de franchissement sur la fonction `generatecode`. Pour montrer que nous avons compris la demande et le fonctionnement attendu du compilateur, nous avons donc traduit par des affichages (`printf`), le code qui devra être généré, comme l'illustre la figure ci-dessous :

Projet Informatique Electronique

```
etape=en;
printf("#\ttest de franchissement de la transition t0\n");
while (etape!=NULL)
{
    printf("\tpush\t16\n");
    printf("\tpushi\t1\n");
    printf("\teq\n");
    printf("\tpush\t1\n");
    printf("\tpush\t9\n");
    printf("\tpushi\t128\n");
    printf("\tgt\n");
    printf("\tand\n");
    printf("\tand\n");
    printf("\tjtf\tnext0000\n");
    printf("\tpushi\t1\n");
    printf("\tpop\t25\n");
    printf("\tpushi\t1\n");
    printf("\tpop\t28\n");
}
```

Figure 5 Partie du compilateur

Ce code devra, permettre de permettre de générer (d'afficher) le code ci-dessous:

```
#test de franchissement de la transition t0
    push    16
    pushi   1
    eq
    push    1
    push    9
    pushi   128
    ls
    and
    and
    jf next0000
    pushi   1
    pop 25
    pushi   1
    pop 28
next0000:
```

Figure 6 Code en assembleur

Nous avons fait de même pour les trois autres tests de franchissement à savoir :

#test de franchissement de la transition t1

#test de franchissement de la transition t2

#test de franchissement de la transition t3

Projet Informatique Electronique

Ensuite nous avons essayé d'implémenter la machine virtuelle en dure puis par liaison série dans la carte SMT32F0, cela n'a pas été productif, même après de nombreux essais astucieux. Sachant que le problème ne vient pas de la machine virtuelle, il doit probablement s'agir d'une mauvaise configuration des pattes de la carte ou d'un manque de paramètres nécessaire pour l'exécution de la machine virtuelle par la carte.

Pour la commande du moteur, nous avons réussi à générer par une patte de la nucléo, une PWM en entrée du driver et de la récupérer (un peu déformée) en sortie du driver en réalisant un câblage.

Nous n'avons pas fait la lecture des capteurs et le grafcet de pilotage du moteur en fonction des informations des capteurs.

Pour conclure

En sommes, ce projet a été très intéressant et enrichissant du fait de la combinaison des deux modules clés de notre formation mais aussi et surtout du fait des connaissances et aptitudes que nous avons pu développer en langage C, en microcontrôleur, en gestion de projet sur la durée, en travail d'équipe et en autonomie.